

CCAR-F 쉬운 요약 교재

도메인별로 "이것만은 알고 들어가자"를 쉽게 정리한 입문 교재입니다. 가이드 PDF가 판단 원칙 중심이라면, 이 교재는 용어와 동작 방식을 처음 보는 사람도 이해할 수 있게 풀어 썼습니다.

각 장의 구성: 한 줄 요약 → 쉬운 비유 → 핵심 개념 → 헷갈리는 비교 → 시험 합정 → 30초 셀프체크

2026-10 기준, Anthropic 공식 문서(platform.claude.com, code.claude.com)로 사실관계를 확인했습니다. 제품은 자주 바뀌므로 시험 직전에 공식 문서를 한 번 더 보세요.

0. 시험 전체를 관통하는 한 문장

"요구사항을 만족하는 가장 작고, 가장 안전하고, 가장 예측 가능한 설계를 골라라."

시험 선택지에는 늘 "멋있어 보이는 답"(agent 추가, 모델 교체, reviewer 추가, router 추가)이 섞여 있습니다. 대부분 오답입니다. 먼저 이렇게 자문하세요.

질문	예라면
프롬프트·설명·schema만 고쳐서 해결되나?	그게 정답일 확률이 높다
반드시 지켜져야 하는 규칙인가?	프롬프트가 아니라 코드(hook·권한)로 강제
되돌릴 수 없는 행동인가?	실행 직전에 승인/검증 지점
작업끼리 서로 기다려야 하나?	기다리면 순차, 아니면 병렬
기계가 결과를 읽나?	자유 텍스트 말고 schema

1. Domain 1 — Agentic Architecture & Orchestration (27%)

한 줄 요약

Agent는 "모델이 도구를 쓰면서 스스로 다음 할 일을 정하는 반복 루프"이고, 시험은 "언제 agent를 쓰고, 언제 쓰지 말고, 여러 개를 어떻게 엮는가"를 묻습니다.

쉬운 비유

- **고정 workflow** = 공장 컨베이어 벨트. 순서가 정해져 있고 매번 똑같다.
- **Single agent** = 혼자 일하는 숙련공. 상황 보고 알아서 판단한다.
- **Coordinator + subagents** = 팀장과 팀원. 팀장은 일을 나눠주고 결과만 받아 합친다. 팀원은 자기 일에 필요한 자료만 받는다.

1.1 Agentic loop 동작 방식

Messages API로 직접 구현하면 이렇게 됩니다.

1. 모델 호출 (사용 가능한 tools 목록과 함께)
2. 응답의 `stop_reason` 확인
3. `"tool_use"` → 모델이 도구를 쓰고 싶어함. 내 코드가 도구를 실행하고, 결과를 `tool_result` 블록에 담아 **user 메시지로** 다시 보냄 → 1로
4. `"end_turn"` → 모델이 할 일을 끝냄. 루프 종료
5. 안전장치: 최대 turn 수, 시간, 비용 한도

핵심: 루프 자체는 쉽습니다. 시험이 보는 건 **종료 조건과 실패 처리**입니다. "언제 멈추나? 도구가 실패하면? 위험한 행동 전엔?"

1.2 구조 선택표

구조	언제	단점
단일 모델 호출	한 번에 답할 수 있음	복잡한 일엔 부족
고정 workflow (prompt chaining)	단계와 순서가 항상 같음	유연성 낮음 (대신 안정적)
Single agent	단계가 서로 강하게 얽혀 있고 상태 공유가 중요	context가 비대해짐
Coordinator + subagents	독립된 작업 여러 개, 전문화, context 격리가 이득	분배·병합·실패 관리 비용

기본값은 위에서부터. 아래로 갈수록 "그래야 할 이유"가 분명해야 합니다.

1.3 병렬 vs 순차

판단 기준은 딱 하나, **의존성**입니다.

- 앞 단계 결과가 다음 단계 입력을 바꾼다 → **순차**
- 서로 결과를 기다릴 필요 없이 합치기만 하면 된다 → **병렬**
- 같은 파일·리소스를 동시에 고친다 → 병렬 금지 (순차 또는 소유권 분리)
- 실무에서 흔한 건 **hybrid**: 1단계(순차) → 여러 갈래 병렬 조사 → coordinator가 병합

1.4 Subagent에게 무엇을 주고 받나

- **줄 때:** 그 일에 필요한 지시와 입력만. 팀장의 전체 대화 기록을 통째로 넘기지 않는다.
- **받을 때:** 원문 대화 전체가 아니라 **압축된 구조화 결과** (결론, 근거/출처, 상태).
- **권한:** 읽기만 하면 되는 subagent엔 읽기 도구만. 자격증명 공유 금지.

subagent를 쓰는 진짜 이유는 "여러 명이어서"가 아니라 **context를 깨끗하게 분리**할 수 있어서입니다.

1.5 Hooks — "부탁"이 아니라 "강제"

Hook은 특정 시점에 **내 코드가 자동 실행**되게 하는 장치입니다. 모델의 판단과 무관하게 항상 실행됩니다.

자주 나오는 이벤트	용도
PreToolUse	도구 실행 전 검사·차단·입력 수정 (예: /src 밖 쓰기 차단)
PostToolUse	도구 실행 후 기록·후처리 (예: 감사 로그, 포맷터 실행)
UserPromptSubmit	사용자 입력이 모델에 가기 전 검사·보강
Stop	모델이 멈추려 할 때 조건 확인 (예: 테스트 통과 전엔 못 멈춤)
SessionStart	세션 시작 시 컨텍스트 주입

Claude Code에서 command hook이 **exit code 2**로 끝나면 해당 동작이 차단됩니다. PreToolUse에서는 JSON으로 `permissionDecision: "deny"`를 돌려줘도 막힙니다.

시험 공식: 보안·정책·감사·비가역 행동 = hook/권한(결정적 강제). "프롬프트에 하지 말라고 쓰기"는 오답.

1.6 Session: Resume / Fork / Checkpoint

- **Resume(continue):** 같은 작업을 이어서 한다. 맥락 그대로.
- **Fork:** 지금 상태는 보존하고, **복사본에서** 다른 가설을 시험한다. (게임 세이브 파일 복사)
- **Checkpoint:** 긴 작업 중간 결과를 바깥에 저장해, 실패해도 처음부터 다시 하지 않게 한다.

시험 합정

- 절차가 고정인데 multi-agent를 고르는 것 (과설계)
- "agent가 알아서 판단하게" → 반드시 지켜야 하는 규칙인데도
- 모든 걸 병렬로 → 의존성 무시

30초 셀프체크

- ☐ stop_reason이 tool_use일 때 내 코드가 할 일 3가지를 말할 수 있다
- ☐ 고정 workflow가 agent보다 나은 상황 1개를 들 수 있다
- ☐ PreToolUse와 PostToolUse의 용도 차이를 말할 수 있다
- ☐ fork와 resume의 차이를 비유로 설명할 수 있다

2. Domain 2 — Tool Design & MCP Integration (18%)

한 줄 요약

모델에게 tool의 이름·설명·schema는 사용설명서이자 길안내 표지판입니다. 도구 문제는 대부분 이 셋을 고쳐서 풀니다.

쉬운 비유

신입사원에게 서랍 20개를 주면서 이름표를 "서랍1, 서랍2..."로 붙였다고 생각해 보세요. 엉뚱한 서랍을 여는 건 신입 탓이 아니라 이름표 탓입니다. 감시원(classifier)을 붙이기 전에 **이름표부터 고칩니다**.

2.1 좋은 tool의 체크리스트

- ☐ 이름이 동작을 말한다: `get_order` (O), `order_tool` (X)
- ☐ 설명에 "언제 쓰는지 / 언제 쓰지 않는지"가 있다
 - 예: "정확한 주문번호가 있을 때 사용. 이름·날짜로 찾을 땐 `search_orders` 사용"
- ☐ input schema: 필수/선택 구분, 타입, enum, 형식 예시
- ☐ 결과가 파싱 가능한 구조
- ☐ 실패도 구조화해서 돌려준다
- ☐ 한 도구가 너무 많은 일을 하지 않는다 (특히 읽기와 쓰기를 섞지 않는다)

2.2 에러는 "다음 행동을 정할 수 있게" 돌려준다

API에서 도구 실패는 `tool_result`에 `is_error: true`를 붙여 모델에게 돌려줍니다. 루프를 죽이거나 빈 결과를 주면 모델이 복구할 방법이 없습니다.

나쁜 예: `"failed"`

좋은 예:

```
{ "error_type": "RATE_LIMITED", "retryable": true,
  "retry_after_s": 30, "message": "upstream API rate limit",
  "suggested_action": "wait and retry" }
```

`VALIDATION_ERROR` (입력이 틀림 → 재시도 무의미, 입력 수정)와 `TIMEOUT` (일시적 → 재시도 가능)을 구분하는 게 핵심입니다.

2.3 Tool이 너무 많을 때

도구가 많을수록 고르기 어렵고 context도 많이 씁니다. 첫 조치는 **역할별로 필요한 도구만 노출**하는 것. 모델 교체나 예시 추가보다 먼저입니다.

2.4 MCP (Model Context Protocol)

한 줄: AI 앱이 외부 시스템(DB, Jira, GitHub, 사내 solver...)에 연결하는 방식을 표준화한 프로토콜. "AI용 USB-C"라고 흔히 비유합니다.

MCP server가 제공하는 것:

- **Tools:** 모델이 호출하는 기능 (예: `create_issue`)

- **Resources:** 읽어올 수 있는 데이터 (예: 파일, 레코드)
- **Prompts:** 재사용 프롬프트 템플릿

Claude Code에서 MCP 설정 위치:

범위	위치	용도
Project	repo 루트의 <code>.mcp.json</code> (커밋)	팀 전체 공유
User	<code>~/.claude.json</code> (<code>claude mcp add --scope user</code>)	나만 쓰는 서버

비밀값(토큰)은 `.mcp.json`에 직접 쓰지 말고 `${JIRA_TOKEN}` 같은 **환경변수 참조**로.

시험이 보는 것은 프로토콜 세부가 아니라 **"무엇을 MCP로 노출할지, 권한·인증을 어떻게 줘줄지"**입니다.

2.5 Built-in vs Custom

플랫폼이 이미 제공하는 기능으로 충분하면 직접 만들지 않습니다. 사내 시스템, 비즈니스 규칙, 특수 검증이 필요할 때 custom tool/MCP.

시험 함정

- 도구 선택 오류 → "classifier 모델 추가" (오답, 설명부터)
- 에러를 문자열 하나로 뭉뚱그림
- 읽기 전용 분석 agent에게 쓰기 도구까지 전부 노출

30초 셀프체크

- ☐ 비슷한 두 도구를 구분하는 description을 한 줄씩 쓸 수 있다
- ☐ `is_error: true`를 언제 쓰는지 안다
- ☐ retryable 오류와 아닌 오류 예를 하나씩 든다
- ☐ 팀 공유 MCP 설정이 어디 들어가는지 안다

3. Domain 3 — Claude Code Configuration & Workflows (20%)

한 줄 요약

Claude Code는 "어디에 무엇을 적느냐"로 동작이 결정됩니다. 지침(CLAUDE.md), 설정(settings.json), 확장(skill·subagent·hook)을 구분하는 문제가 핵심입니다. 이 영역은 암기 비중이 큼니다.

쉬운 비유

- **CLAUDE.md** = 새로 온 동료에게 주는 "우리 팀 업무 메모"
- **settings.json (permissions)** = 출입 카드 권한
- **Hook** = 출입문 보안 게이트 (사람 판단과 무관하게 작동)
- **Skill** = 필요할 때 꺼내 보는 업무 매뉴얼 바인더
- **Subagent** = 별도 방에서 일하는 전문가 (자기만의 context)

3.1 CLAUDE.md

매 세션 시작 시 자동으로 읽히는 프로젝트 지침입니다.

넣을 것:

- 빌드·테스트 명령 (`pnpm test --filter api`)
- 코딩 규칙, 폴더 구조, 아키텍처 관례
- 주의할 점

넣지 말 것:

- 반드시 막아야 하는 것 → **permission deny / hook / managed policy**
- 가끔만 필요한 긴 절차 → **skill** (매 세션 context 낭비)

위치: 프로젝트 루트 CLAUDE.md (팀 공유), `~/.claude/CLAUDE.md` (개인 전체). 길어지면 `.claude/rules/` 로 주제별 분리(경로별 적용 가능).

3.2 Settings 우선순위 ★암기

위가 이깁니다.

순위	범위	파일
1	Managed (조직 정책)	managed-settings.json / MDM / 관리 콘솔
2	Command line	<code>claude --settings ...</code> 등 실행 인자
3	Project local (나만, 이 프로젝트)	<code>.claude/settings.local.json</code> (gitignore)
4	Shared project (팀 공유)	<code>.claude/settings.json</code> (커밋)
5	User (나, 모든 프로젝트)	<code>~/.claude/settings.json</code>

외우는 법: "회사 > 지금 > 나만 > 우리 팀 > 내 기본값"

3.3 Permission 규칙

- 규칙 종류: `allow`, `ask`, `deny`

- 평가 순서: **deny** → **ask** → **allow**, 먼저 걸리는 것이 결정
- 넓은 deny가 좁은 allow보다 강합니다. `deny: Bash(aws *)` 가 있으면 `allow: Bash(aws s3 ls)` 로 예외를 만들 수 없습니다.
- `.env` 같은 민감 파일은 `Read(./ .env)` deny 규칙으로 막습니다.

3.4 Plan mode

파일을 고치기 전에 **탐색과 계획만** 하게 하는 모드. 계획을 검토·승인한 뒤 실행합니다. 쓸 때: 큰 변경, 낯선 repo, 위험한 작업.

3.5 확장 기능 구분표 ★자주 출제

기능	무엇	언제 로드/실행	위치
CLAUDE.md	상시 프로젝트 지침	매 세션 자동	프로젝트 루트
Skill	재사용 절차·지식 묶음 (SKILL.md + 보조 파일)	관련 있을 때 자동, 또는 /이름 으로 호출	<code>.claude/skills/<이름>/</code>
Command	/이름 으로 부르는 단일 프롬프트 (현재는 skill과 같은 메커니즘)	사용자가 호출	<code>.claude/commands/</code>
Hook	이벤트 시점에 코드 실행·강제	이벤트 발생 시 항상	settings.json의 hooks
Subagent	독립 context·역할·도구 제한을 가진 별도 agent	위임할 때	<code>.claude/agents/<이름>.md</code>

판단 요령:

- "가끔 필요, 필요할 때만 불러와" → **Skill**
- "반드시, 예외 없이" → **Hook / permission**
- "메인 context를 더럽히지 않게 따로" → **Subagent**
- "항상 알아야 하는 프로젝트 맥락" → **CLAUDE.md**

3.6 CI/CD와 non-interactive 실행

CI에선 사람이 답할 수 없으므로 **결정적으로** 돌려야 합니다.

옵션	역할
<code>claude -p "..."</code>	non-interactive(print) 모드
<code>--output-format json</code>	결과를 JSON으로 (파싱용). <code>stream-json</code> 은 실시간 스트림
<code>--json-schema '<schema>'</code>	지정한 schema대로 결과 (<code>structured_output</code> 필드)
<code>--allowedTools "Read,Edit"</code>	문지 않고 쓸 도구를 명시적으로 제한
<code>--max-turns N</code>	반복 횟수 제한
<code>--permission-mode</code>	세션 권한 기본값 (예: <code>dontAsk</code> 는 묻는 대신 거부)
<code>--continue / --resume <id></code>	이전 세션 이어가기
<code>--bare</code>	로컬 hook·plugin·CLAUDE.md 자동 로드 생략 → 어느 머신에서나 같은 결과

종료 코드: 성공 0, 실패 non-zero → 파이프라인에서 분기 가능.

시험 합정

- 금지사항을 CLAUDE.md에만 적음 (강제 안 됨)
- 개인 실험 설정을 팀 공유 파일에 넣음 → `settings.local.json` 으로
- 하위 설정이 조직 정책을 뒤집을 수 있다고 착각
- CI에서 interactive 모드 사용

30초 셀프체크

- ☐ 설정 우선순위 5단계를 순서대로 말할 수 있다
- ☐ deny/ask/allow 평가 순서를 안다
- ☐ skill / hook / subagent를 한 문장씩 구분한다
- ☐ CI용 명령어 한 줄을 직접 쓸 수 있다

4. Domain 4 — Prompt Engineering & Structured Output (20%)

한 줄 요약

명확하게 말하고, 예시로 보여주고, 기계가 읽을 결과면 schema로 묶고, 검증 실패는 이유를 알려주며 제한적으로 재시도.

쉬운 비유

외주 업체에 일을 맡긴다고 생각하세요. "알아서 잘"이라고 하면 결과가 들쭉날쭉합니다. 목적·조건·완성 기준·제출 양식을 주고, 잘된 예시 2~3개를 보여주면 결과가 안정됩니다.

4.1 명확한 지시

Claude는 명확하고 직접적인 지시에 잘 반응합니다. 다음을 적으세요.

- **목표:** 무엇을 위해
- **제약:** 하지 말 것, 범위
- **성공 기준:** 무엇이면 완료인가
- **출력 형식:** 어떤 모양으로
- **맥락/이유:** 왜 그 규칙이 있는지 (이유를 알면 더 잘 일반화)

4.2 Few-shot

규칙을 길게 설명하는 것보다 **좋은 입출력 예시 몇 개**가 효과적일 때가 많습니다.

- 특히 **경계 사례**(애매한 케이스)를 예시에 넣는다
- 예시는 그대로 따라 하므로 나쁜 습관이 섞이지 않게
- 예시끼리 다양하게 (모두 비슷하면 그 패턴만 배움)

4.3 Structured output — 언제 쓰나

아래 중 하나라도 해당하면 자유 텍스트 대신 **JSON schema** 기반:

- 애플리케이션이 결과를 파싱한다
- DB/API로 넘긴다
- 필수 필드 누락이 치명적이다
- enum/숫자/배열 타입을 보장해야 한다

방법: tool의 `input_schema`로 형식을 정의해 그 도구를 쓰게 하거나(`tool_choice`로 특정 도구 강제), structured outputs(JSON schema) 기능을 사용. Claude Code CI에선 `--json-schema`.

regex로 자유 텍스트를 긁어오는 건 증상 처리, schema가 근본 해결입니다.

4.4 Validation + 재시도

1. 결과를 schema/비즈니스 규칙으로 검증
2. 실패 시 **무엇이 왜 틀렸는지**를 모델에게 알려주고 재시도 (targeted retry)
3. 재시도 횟수 **제한**
4. 그래도 실패하면 fallback 또는 사람 검토

금지: 같은 요청 무한 반복, 실패 건 조용히 버리기.

4.5 Multi-pass review (생성 → 검토 분리)

품질을 올리지만 비용·지연·복잡성도 올립니다.

- 쓸 때: **오류 비용이 크고** 검토 기준을 **명시할 수 있을 때** (계약서 조항 추출 등)
- 안 쓸 때: 기본값으로 모든 요청에

4.6 프롬프트 개선 순서 ★

1. 요구사항·성공 기준 명확히
2. 출력 형식/schema 지정
3. 좋은 예시 추가
4. 작업 분해
5. 검증·재시도
6. (정말 필요할 때만) reviewer agent / 모델 교체

선택지에 "다른 모델을 하나 더 붙여 검증"이 있으면, 더 작은 수정(1~3번)이 있는지 먼저 보세요.

시험 합정

- 형식이 흔들림 → temperature만 낮추기 (부분적 효과, 근본 해결 아님)
- 경계 사례 불일치 → 모델 교체 (few-shot이 먼저)
- 모든 출력에 reviewer

30초 셀프체크

- ☐ structured output이 필요한 조건 4가지 중 3개를 말한다
- ☐ targeted retry가 무엇인지 설명한다
- ☐ few-shot 예시에 무엇을 넣어야 하는지 안다
- ☐ 프롬프트 개선 6단계 순서를 안다

5. Domain 5 — Context Management & Reliability (15%)

한 줄 요약

context window는 무한 메모리가 아니라 비싼 작업대입니다. 필요한 것만 올리고, 오래된 건 요약하고, 진행 상황은 바깥에 저장하고, 실패는 정상 상황으로 설계합니다.

쉬운 비유

책상(context)이 클수록 좋을 것 같지만, 서류를 계속 쌓으면 중요한 서류가 묻히고 찾는 데 시간도 돈도 듭니다. 지금 일에 필요한 서류만 올리고, 끝난 건 요약 메모로 바꾸고, 진행표는 벽(외부 파일)에 붙여 둡니다.

5.1 Context 전략 5가지

전략	무엇	언제
Selective context	현재 작업에 필요한 정보만 넣기	항상
Summarization	오래된 기록을 결정사항·미해결 중심으로 압축	긴 대화
Checkpoint / 외부 state	진행 상황·결정을 파일/DB에 저장	긴 작업, 압축으로 잊어버릴 때
Prompt caching	반복되는 긴 앞부분을 재사용	같은 prefix를 반복 호출
Subagent isolation	독립 작업을 별도 context로	조사 결과가 메인을 오염시킬 때

큰 원본 데이터는 context에 통째로 넣지 말고 **도구로 필요한 부분만 조회**합니다.

5.2 Prompt caching ★숫자 포함

- 캐시는 **prefix(앞부분)** 일치 기반. 순서: **tools → system → messages**
- 따라서 **변하지 않는 것(도구 정의, 정책 문서)을 앞에**, 매번 바뀌는 질문은 뒤에
- 고정 부분 끝에 `cache_control: {"type": "ephemeral"}` 로 breakpoint 표시 (요청당 최대 4개)
- TTL: 기본 **5분**(사용할 때마다 갱신), 옵션 **1시간**(`"ttl": "1h"`)
- 비용: 캐시 쓰기는 기본 입력 단가보다 싸고(5분 1.25배, 1시간 2배), **캐시 읽기는 훨씬 싼**(기본 0.1배 수준)
- 앞부분이 바뀌면 그 뒤 캐시는 무효 (예: tool 정의를 바꾸면 전부 무효)
- 너무 짧은 프롬프트는 캐시되지 않음 (모델별 최소 길이 있음)

5.3 모호함: 물어볼까, 진행할까

- 결과를 크게 바꾸고, 잘못 추정하면 재작업이 큰 모호함 → **시작 전에 확인**
- 영향이 작고 되돌릴 수 있는 것 → 합리적으로 가정하고 진행, 가정은 명시
- 모든 걸 물어보면 자동화 가치가 사라짐

5.4 신뢰성 체크리스트

- ☐ 재시도 가능/불가능 오류 구분 (타임아웃 vs 입력 오류)
- ☐ **Idempotency(멱등성)**: 같은 요청을 두 번 보내도 한 번만 실행되게. 결제·환불은 idempotency key + 실행 전 상태 조회
- ☐ **부분 성공 표현**: "10건 중 8건 성공, 2건 실패(사유)"

- ☐ Timeout / max turns / 예산 한도
- ☐ Provenance·감사 기록: 무엇을 근거로 무엇을 했는지
- ☐ Human review 경계: 영향이 큰 행동 앞에만

시험 함정

- context가 길어 문제 → "더 큰 모델" (요약·state 분리가 먼저)
- 결제 타임아웃 → 즉시 재시도 (이중 결제 위험)
- 긴 작업 중 진행 상황을 context에만 의존

30초 셀프체크

- ☐ 캐싱을 위해 프롬프트 순서를 어떻게 배치하는지 안다
- ☐ 캐시 TTL 두 가지를 안다
- ☐ idempotency key가 왜 필요한지 예로 설명한다
- ☐ 긴 작업에서 외부 state 파일에 무엇을 적을지 말한다

6. 한 장 비교표 — 시험에서 자주 맞붙는 쌍

보통 오답 쪽	보통 정답 쪽	판단 기준
프롬프트에 "하지 마"	Hook / permission / managed policy	반드시 지켜야 하나?
Multi-agent	고정 workflow / single agent	정말 독립·격리가 필요한가?
전부 병렬	의존성에 따라 순차/병렬	결과를 기다려야 하나?
자유 텍스트 + regex	Structured output (schema)	기계가 읽나?
전체 history 유지	요약 + 외부 state	길고 오래 가나?
Classifier / router 추가	Tool 이름·설명·schema 개선	더 작은 수정이 있나?
자동 실행	승인 지점 (비가역 행동 앞)	되돌릴 수 있나?
무조건 재시도	오류 유형별 대응 + 횟수 제한	재시도로 해결되는 오류인가?
더 큰 모델	프롬프트·예시·schema 개선	모델 문제가 맞나?
CLAUDE.md에 긴 절차	Skill	항상 필요한가, 가끔인가?
팀 파일에 개인 설정	settings.local.json	누가 쓰는 설정인가?

7. 용어집 (영어 시험 대비)

용어	뜻
agentic loop	모델→도구 실행→결과 관찰→다음 행동의 반복
orchestration	여러 단계/agent를 엮어 전체 흐름을 조율
coordinator / orchestrator	일을 나누고 결과를 합치는 상위 agent
subagent	위임받은 일을 별도 context에서 수행하는 하위 agent
handoff	작업과 맥락을 다른 agent에 넘기는 것
fan-out / fan-in	여러 갈래로 나눠 병렬 처리 / 결과를 모음
deterministic enforcement	모델 판단과 무관하게 코드로 항상 강제
least privilege	필요한 최소 권한만 부여
blast radius	실패나 실수가 미치는 피해 범위
human-in-the-loop	특정 지점에서 사람이 승인·검토
idempotency	여러 번 실행해도 결과가 한 번 실행과 같음
provenance	결과의 출처·근거 기록
prompt chaining	한 호출의 출력을 다음 호출의 입력으로 잇는 고정 흐름
few-shot	프롬프트에 입출력 예시 몇 개를 넣는 방식
structured output	schema에 맞춘 기계 판독용 출력
targeted retry	실패 원인을 알려주며 하는 제한적 재시도
context window	모델이 한 번에 볼 수 있는 입력 범위
compaction	긴 대화를 요약해 context를 줄이는 것
prompt caching	반복되는 prompt 앞부분을 캐시해 비용·지연 절감
MCP	외부 도구·데이터 연결 표준 프로토콜
headless / non-interactive	사람 입력 없이 실행 (CI 등)
plan mode	수정 없이 탐색·계획만 하는 모드
managed policy	조직 관리자가 강제하는 최상위 설정

참고 (공식 문서)

- Prompt caching: platform.claude.com/docs/en/build-with-claude/prompt-caching
- Tool use: platform.claude.com/docs/en/agents-and-tools/tool-use/overview
- Claude Code settings: code.claude.com/docs/en/settings
- Claude Code permissions: code.claude.com/docs/en/permissions
- Claude Code hooks: code.claude.com/docs/en/hooks
- Claude Code headless (`claude -p`): code.claude.com/docs/en/headless
- `.claude` 디렉터리 구조: code.claude.com/docs/en/claude-directory